

Lecture 14

Bootstrapping

SEC 595: Encrypted Computing

Roadmap

BFV, BGV and CKKS (Lectures 10–12) are levelled: the depth they support is fixed at key generation. This lecture removes that limit with one idea — *evaluate the scheme's own decryption under encryption*. Section 1 states the limit and observes that decryption is a circuit. Section 2 gives the bootstrapping construction and proves it correct and refreshing. Section 3 states when a scheme is bootstrappable and works three examples. Section 4 treats the extra assumption the construction needs, circular security.

1. The Ceiling

1.1 Every Scheme So Far Is Levelled

In the lattice schemes every ciphertext carries noise. Addition adds noises and multiplication multiplies them, and decryption fails once the noise passes a threshold. So the multiplicative depth L is fixed at KeyGen (Lecture 8, §1.6: LHE).

A ciphertext therefore has a **history**, and whether it can still be used depends on that history, not on the message: two ciphertexts of the same m may be fresh and exhausted respectively. A larger modulus raises the ceiling; it never removes it.

1.2 Decryption Is a Circuit

Decryption of a scheme $\mathcal{E}$ is a function of two arguments,

$$\text{Dec}_{\mathcal{E}}(\text{Sk}, \text{ct}) = m.$$

It is fixed, public and finite, and the same for every ciphertext. So it is a valid choice of the function Π in

$$\text{Evaluate}(\text{Pk}, \Pi, \text{ct}_1, \dots, \text{ct}_k).$$

The only obstacle is that Evaluate needs its inputs *encrypted*, and one of the inputs is the secret key. Nothing in the signature of Evaluate restricts what the input ciphertexts encrypt, so the obstacle is removed directly: encrypt the secret key.

2. Recryption

2.1 The Two Ingredients

Definition 1 (Bootstrapping key; encrypted ciphertext). Let $(\text{Pk}, \text{Sk}) \leftarrow \text{KeyGen}$.

- The **bootstrapping key** is $\overline{\text{Sk}} = \text{Enc}(\text{Pk}, \text{Sk})$. It is computed once at setup and published with Pk.
- For a ciphertext ct to be refreshed, the **encrypted ciphertext** is $\overline{\text{ct}} = \text{Enc}(\text{Pk}, \text{ct})$.

A ciphertext is a string (a vector, a pair of polynomials, a matrix), so it can be a plaintext. In practice Sk and ct are encrypted piece by piece, for instance bit by bit. $\overline{\text{ct}}$ has two layers: an *outer* one we compute under, and an *inner* one we want to remove. Publishing $\overline{\text{Sk}}$ gives no one the ability to decrypt; only Evaluate can use it.

2.2 Evaluating Decryption Under Encryption

Definition 2 (Bootstrapping, Gentry 2009).

$$\text{ct}^\# = \text{Evaluate}(\text{Pk}, \text{Dec}_\mathcal{E}, \overline{\text{ct}}, \overline{\text{Sk}}).$$

No new algorithm is introduced: bootstrapping is Evaluate called with $\Pi = \text{Dec}_\mathcal{E}$ on the inputs $\overline{\text{ct}}, \overline{\text{Sk}}$. The whole procedure is

$$\begin{aligned} (\text{Pk}, \text{Sk}) &\leftarrow \text{KeyGen}, & \overline{\text{Sk}} &\leftarrow \text{Enc}(\text{Pk}, \text{Sk}) & (\text{once}) \\ \overline{\text{ct}} &\leftarrow \text{Enc}(\text{Pk}, \text{ct}), & \text{ct}^\# &\leftarrow \text{Evaluate}(\text{Pk}, \text{Dec}_\mathcal{E}, \overline{\text{ct}}, \overline{\text{Sk}}) & (\text{per refresh}). \end{aligned}$$

Proposition 1 (Correctness). *If $\mathcal{E}$ is correct for $\text{Dec}_\mathcal{E}$ and ct decrypts to m , then $\text{ct}^\#$ decrypts to m .*

Proof. By correctness property 2 (Lecture 8, §1.2) with $\Pi = \text{Dec}_\mathcal{E}$ and inputs $\overline{\text{ct}}, \overline{\text{Sk}}$, the output decrypts to $\text{Dec}_\mathcal{E}(\text{Sk}, \text{ct})$, which is m by property 1. $\square$

The inner layer is stripped and the outer one stays. At no point is m in the clear, and the evaluator holds only Pk, $\overline{\text{ct}}$ and $\overline{\text{Sk}}$, never Sk. The hypothesis “correct for $\text{Dec}_\mathcal{E}$ ” is not free; Section 3 is about when it holds.

2.3 Why the Result Is Refreshed

A map sending a ciphertext of m to another ciphertext of m is worthless by itself; the identity does that. What matters is the noise of the output.

Proposition 2 (Refreshing). *The noise of $\text{ct}^\#$ depends only on the circuit $\text{Dec}_\mathcal{E}$, not on the history of ct.*

Proof sketch. $\text{ct}^\#$ is the output of one run of Evaluate on the fixed circuit $\text{Dec}_\mathcal{E}$, applied to the fresh ciphertexts $\overline{\text{ct}}$ and $\overline{\text{Sk}}$. The noise of ct is inside the *plaintext* of $\overline{\text{ct}}$, and is removed by the decryption being evaluated; it never enters the noise of the outer layer. $\square$

Compactness (Lecture 8, §1.4) is what makes $\text{Dec}_\mathcal{E}$ fixed: ct has size bounded in λ , so the circuit that decrypts it has fixed size and depth. And $\text{ct}^\#$ is an ordinary ciphertext, so compactness is preserved. A worn-out ciphertext returns at a fixed, known noise level.

3. Bootstrappable Schemes

3.1 The Condition

Measure noise in levels. A fresh ciphertext supports D levels of multiplication; evaluating $\text{Dec}_{\mathcal{E}}$ consumes d of them.

Definition 3 (Bootstrappable). $\mathcal{E}$ is **bootstrappable** if it can evaluate its own decryption circuit plus at least one more operation:

$$d < D.$$

Proposition 3. *If $\mathcal{E}$ is bootstrappable, then $\mathcal{E}$ with bootstrapping evaluates circuits of unbounded depth: levelled + bootstrapping = FHE.*

Proof sketch. By the refreshing proposition, every bootstrapped ciphertext has the same $D - d \geq 1$ levels left, whatever came before. Alternate evaluate $\rightarrow$ bootstrap $\rightarrow$ evaluate $\rightarrow \dots$; each round makes progress of at least one level, so any finite depth is reached. $\square$

3.2 Depth Is the Currency

The budget is spent by *multiplicative depth*, not gate count: additions grow the noise by a summand, multiplications by a factor, so only the longest chain of multiplications matters. Hence $\text{Dec}_{\mathcal{E}}$ must be *shallow*, however wide. Lattice decryption — an inner product and a rounding test — is many additions and little depth.

The difficulty is that D and d grow together: larger parameters raise D , but they also enlarge the ciphertext and so the decryption circuit, raising d . Bootstrapping is correct but expensive; in practice it dominates the cost of a computation.

3.3 Examples

Example 1 (Where the budget goes). A fresh ciphertext supports $D = 10$ levels; $\text{Dec}_{\mathcal{E}}$ costs $d = 7$.

(a) *Levels per bootstrap.* Each bootstrap returns a ciphertext with $D - d = 10 - 7 = 3$ useful levels.

(b) *Depth 100.* The fresh input gives $D = 10$ levels; each bootstrap adds 3. With 30 bootstraps,

$$D + 30(D - d) = 10 + 30 \cdot 3 = 100 \geq 100,$$

while with 29,

$$D + 29(D - d) = 10 + 29 \cdot 3 = 97 < 100.$$

So exactly 30 bootstraps are needed.

(c) *Depth L .* The number of bootstraps is the least k with $D + k(D - d) \geq L$:

$$k = \left\lceil \frac{L - D}{D - d} \right\rceil \quad (L > D),$$

and $k = 0$ for $L \leq D$. Check: $\lceil (100 - 10)/3 \rceil = \lceil 90/3 \rceil = 30$.

(d) *Failure.* If $d = D$, a bootstrapped ciphertext has 0 levels left: it can be refreshed again but never used, so no progress is made. If $d > D$, the scheme cannot even evaluate its decryption, and the refreshed ciphertext does not decrypt correctly.

Example 2 (The depth of Regev decryption). Regev decryption (Lecture 7) computes $\Delta = c_2 - c_1^T s \bmod q$ and tests whether Δ is near 0 or near $q/2$.

(a) *Where the multiplications are.* In the products $c_{1,i}s_i$ of the inner product, and in the comparison that implements the rounding test. On bits, adding numbers also needs multiplications (AND gates for the carries).

(b) *Sequence or parallel.* The n products $c_{1,i}s_i$ are independent: they all lie in parallel, at depth one. The sum $\sum_i c_{1,i}s_i$ and the subtraction from c_2 are additions; over $\mathbb{Z}_q$ they cost no depth, and on bits a tree of additions adds only carry chains whose depth grows with $\log n$ and the bit length of q . The rounding test runs on the result, so it lies in sequence after the inner product.

(c) *What dominates.* The rounding test. It is the only non-linear step: the inner product is linear in s with depth one per term, whereas deciding “near 0 or near $q/2$ ” is a comparison on the bits of Δ , a chain of dependent gates.

Example 3 (Bootstrapping BGN). BGN (Lecture 9) supports any number of additions and one multiplication. Its decryption computes C^{q_1} , then a discrete logarithm.

(a) *The recipe.* Steps 1–3 are fine: generate keys, publish $\text{Enc}(\text{Pk}, q_1)$ (encrypted piece by piece), and encrypt C . Step 4 fails. Evaluating Dec_{BGN} requires raising C to the secret exponent q_1 and then a discrete logarithm — many multiplications in sequence — while BGN supports one.

(b) *Structural, not quantitative.* The one multiplication is the pairing $e : \mathbb{G} \times \mathbb{G} \rightarrow \mathbb{G}_1$, and there is no pairing on $\mathbb{G}_1$, so a product cannot be multiplied again. This holds for every choice of parameters: BGN’s limit is structural. Bootstrapping removes only a *noise* ceiling, $d < D$ with D adjustable; it cannot help a scheme whose multiplication count is fixed by its algebra.

4. Circular Security

4.1 Encrypting the Key Under Itself

The bootstrapping key $\overline{\text{Sk}} = \text{Enc}(\text{Pk}, \text{Sk})$ encrypts a secret key under its *own* public key. In the IND-CPA game (Lecture 8, §1.5) the attacker chooses m_0, m_1 without seeing Sk , so the messages are independent of the key. The game says nothing about releasing $\text{Enc}(\text{Pk}, \text{Sk})$: IND-CPA is not violated, but neither does it apply.

Definition 4 (Circular security). $\mathcal{E}$ is **circular secure** if it remains IND-CPA secure when the attacker is also given $\text{Enc}(\text{Pk}, \text{Sk})$.

Circular security is assumed, not proved: no reduction from LWE is known, and no attack either.

4.2 Avoiding the Assumption

Generate a chain of key pairs $(\text{Pk}_0, \text{Sk}_0), \dots, (\text{Pk}_L, \text{Sk}_L)$ and publish

$$\text{Enc}(\text{Pk}_{i+1}, \text{Sk}_i), \quad i = 0, \dots, L-1.$$

Bootstrapping then moves a ciphertext from Pk_i to Pk_{i+1} , and no key is ever encrypted under itself. The price is that only L bootstraps are possible and the public key grows with L : the scheme is levelled again. Unbounded FHE needs circular security.

Summary

1. Levelled schemes have a fixed depth: noise grows with the history of a ciphertext until decryption fails.
2. Bootstrapping is $\text{ct}^\# = \text{Evaluate}(\text{Pk}, \text{Dec}_\mathcal{E}, \text{Enc}(\text{Pk}, \text{ct}), \text{Enc}(\text{Pk}, \text{Sk}))$: decryption evaluated under encryption. No new algorithm is needed.
3. $\text{ct}^\#$ encrypts the same m , with noise depending only on $\text{Dec}_\mathcal{E}$, not on the history of ct ; compactness is preserved.
4. $\mathcal{E}$ is bootstrappable if $d < D$; then levelled becomes fully homomorphic. Depth, not gate count, is the cost, and D, d grow together. Bootstrapping removes noise ceilings only, not structural ones such as BGN's.
5. Publishing $\text{Enc}(\text{Pk}, \text{Sk})$ needs circular security; a key chain avoids it but is levelled.

Exercises

1. Write bootstrapping in the four-algorithm interface of Lecture 8: name Π , the input ciphertexts and the output. Which of the two correctness properties is used where in the correctness proof?
2. With $D = 10$ and $d = 7$ as in Example 1, how many bootstraps does a circuit of depth 40 need? Repeat for depth 100 with $D = 12, d = 8$. Why does the answer not depend on the width of the circuit?
3. Suppose you bootstrap a ciphertext that is already fresh. How do the remaining levels of $\text{ct}^\#$ compare with those of ct ? Explain why bootstrapping after every gate is correct but wasteful.
4. Show that bootstrapping needs compactness: for the non-compact Evaluate of Lecture 8, §1.4, which returns $(\Pi, \text{ct}_1, \dots, \text{ct}_k)$, how does the size of $\text{Dec}_\mathcal{E}$ depend on the history of a ciphertext?
5. For the key chain of Section 4.2, state which key a ciphertext is under before and after the i -th bootstrap, and give the number of published bootstrapping keys as a function of L .
6. State circular security as a game of the same shape as IND-CPA, and mark exactly what the attacker receives that it did not before.